Notes on Machine Learning

Course of Machine Learning

Master Degree in Computer Science

University of Rome "Tor Vergata"

Giorgio Gambosi

Academic Year: 2025-2026

Chapter 1

Generative probabilistic classification

Generative models

In the discussion so far we have mostly emphasized *discriminative* viewpoints, where the focus is on directly separating classes (for instance by learning decision boundaries or posterior probabilities). We now introduce a complementary perspective, the **generative approach**, which is rooted in probabilistic modeling of the data. The guiding idea is simple but powerful: instead of trying to characterize the boundary between classes, we attempt to model *how the data in each class are produced*.

Concretely, each class C_k is described by a suitable *class-conditional distribution* $p(\mathbf{x} \mid C_k)$. This distribution is meant to capture the typical variability of samples belonging to that class. The term "generative" becomes particularly intuitive once we recognize that, if $p(\mathbf{x} \mid C_k)$ is known, then we can in principle *sample* from it and generate random observations that are statistically similar to the training data in class C_k. In the case of text classification, this is directly analogous to the language models encountered earlier: once the model is learned, one can generate random documents that resemble the ones in the collection used to estimate the model.

In addition to the class-conditional distributions, we also introduce the *class priors* $p(C_k)$, which express the probability of observing each class before seeing the data. Priors encode global information about the dataset (for instance, some classes may be more frequent than others). The key point is that, once $p(\mathbf{x} \mid C_k)$ and $p(C_k)$ are available, Bayes' rule provides a direct route to the *posterior* class probabilities $p(C_k \mid \mathbf{x})$, which quantify how likely it is that the observed sample $\mathbf{x}$ belongs to class C_k.

From a practical standpoint, the parameters of both $p(\mathbf{x} \mid C_k)$ and $p(C_k)$ are learned from a labeled dataset, typically by maximum likelihood estimation. Once the posterior probabilities are computed, classification is performed by comparing $p(C_k \mid \mathbf{x})$ across classes and selecting the most probable class. In what follows we derive the posterior probabilities explicitly, first in the binary case and then in the multiclass case, and we show how common choices for $p(\mathbf{x} \mid C_k)$ lead to widely used classifiers.

Deriving posterior probabilities

To make the mechanics of Bayes' rule fully explicit, let us begin with the binary classification case, where the classes are C_1 and C_2. Bayes' rule yields

$$p(C_1|\mathbf{x}) = \frac{p(\mathbf{x}|C_1)p(C_1)}{p(\mathbf{x}|C_1)p(C_1) + p(\mathbf{x}|C_2)p(C_2)} = \frac{1}{1 + \frac{p(\mathbf{x}|C_2)p(C_2)}{p(\mathbf{x}|C_1)p(C_1)}}.$$

This form is already informative: the posterior probability of C_1 depends on a *ratio* between the two joint terms $p(\mathbf{x}|C_k)p(C_k)$. When the numerator dominates, the posterior is close to one; when the denominator dominates, it is close to zero.

A standard and extremely useful reparametrization consists in taking the logarithm of the same ratio. We define

$$a(\mathbf{x}) = \log \frac{p(\mathbf{x}|C_1)p(C_1)}{p(\mathbf{x}|C_2)p(C_2)} = \log \frac{p(C_1|\mathbf{x})}{p(C_2|\mathbf{x})}$$

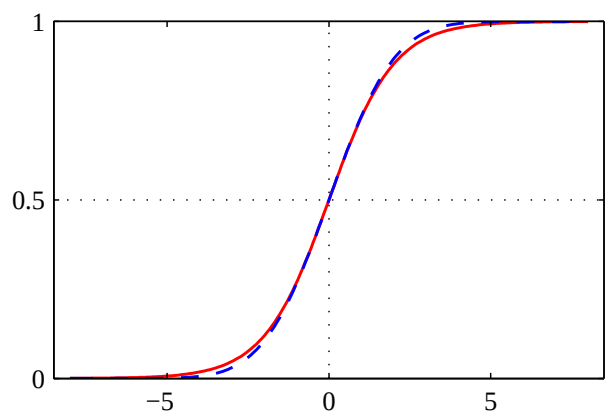
that is, $a(\mathbf{x})$ is the logarithm of the ratio between posterior probabilities. This quantity is known as the **log odds**. The name reflects a precise interpretation: odds measure how many times one event is more likely than another, and the logarithm turns multiplicative comparisons into additive ones, greatly simplifying algebra and optimization.

Substituting $a(\mathbf{x})$ into the previous expression, we obtain the familiar sigmoid form

$$p(C_1|\mathbf{x}) = \frac{1}{1+e^{-a(\mathbf{x})}} = \sigma(a(\mathbf{x})) \qquad p(C_2|\mathbf{x}) = 1 - \frac{1}{1+e^{-a(\mathbf{x})}} = \frac{1}{1+e^{a(\mathbf{x})}}.$$

Here $\sigma(\cdot)$ is the **logistic function**, also called the **sigmoid**. This shows that sigmoid-shaped posterior probabilities arise naturally from Bayes' rule once we express the comparison between classes through the log odds.

Sigmoid



The sigmoid function provides a smooth mapping from $\mathbb{R}$ to $(0, 1)$. In probabilistic terms, it converts an unconstrained real-valued score $a(\mathbf{x})$ (the log odds) into a valid probability. The steepness of the curve around 0 controls how rapidly the classifier transitions from favoring one class to favoring the other; large positive values of $a(\mathbf{x})$ correspond to confident predictions for C_1, while large negative values correspond to confident predictions for C_2.

Two algebraic properties of σ are repeatedly useful:

- $\sigma(-x) = 1 - \sigma(x)$, which mirrors the symmetry between the two classes and is consistent with the identity $p(C_2|\mathbf{x}) = 1 - p(C_1|\mathbf{x})$;

- $\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$, a compact derivative formula that underlies many gradient-based learning procedures.

Deriving posterior probabilities

The binary derivation suggests a natural generalization. When there are $K > 2$ classes, Bayes' rule still provides the posterior probabilities:

$$p(C_k|\mathbf{x}) = \frac{p(\mathbf{x}|C_k)p(C_k)}{\sum_j p(\mathbf{x}|C_j)p(C_j)}$$

The denominator plays the role of a normalization constant ensuring that the posteriors sum to one over k. As in the binary case, it is convenient to introduce log-scores, one for each class. For each $k = 1, \dots, K$, define

$$a_k(\mathbf{x}) = \log(p(\mathbf{x}|C_k)p(C_k)) = \log p(\mathbf{x}|C_k) + \log p(C_k).$$

These $a_k(\mathbf{x})$ can be interpreted as *unnormalized log-posteriors* (up to the same additive constant shared across classes, namely $-\log p(\mathbf{x})$). Writing the posteriors in terms of these scores yields

$$p(C_k|\mathbf{x}) = \frac{e^{a_k}}{\sum_j e^{a_j}} = s(a_k).$$

The mapping $s(\cdot)$ is the **softmax** function (also called the **normalized exponential**). It extends the sigmoid to the multiclass case: it converts a vector of real-valued scores into a probability distribution over classes. Moreover, it can be interpreted as a smoothed version of the maximum: if one score a_k is much larger than all others, then the softmax assigns probability close to one to class k and close to zero to the remaining classes, while still remaining differentiable everywhere.

Gaussian discriminant analysis

The general generative framework becomes concrete once we specify a family of distributions for $p(\mathbf{x} \mid C_k)$. A classical and particularly tractable choice is to assume Gaussian class-conditional distributions, leading to **Gaussian discriminant analysis** (GDA). The Gaussian assumption is attractive for two reasons. First, it often provides a reasonable approximation for real-valued features after suitable preprocessing. Second, it leads to posterior probabilities $p(C_k|\mathbf{x})$ that can be derived analytically and that have a clear geometric interpretation in terms of decision boundaries.

Hypothesis

We first adopt the standard GDA hypothesis that all class-conditional distributions share the same covariance matrix $\boldsymbol{\Sigma}$, of size $d \times d$. Under this assumption,

$$p(\mathbf{x}|C_k) = \frac{1}{(2\pi)^{d/2}|\boldsymbol{\Sigma}|^{1/2}} exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^T \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}_k)\right).$$

The mean vector $\boldsymbol{\mu}_k$ identifies the "center" of class C_k, while $\boldsymbol{\Sigma}$ controls the common spread and correlation structure of the features within classes.

If $K = 2$, we are again in the situation where the posterior probability can be written in sigmoid form:

$$p(C_1|\mathbf{x}) = \sigma(a(\mathbf{x})).$$

The log odds are

$$\begin{aligned} a(\mathbf{x}) &= \log \frac{p(\mathbf{x}|C_1)p(C_1)}{p(\mathbf{x}|C_2)p(C_2)} \\ &= \frac{1}{2}(\boldsymbol{\mu}_2^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_2 - \mathbf{x}^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_2 - \boldsymbol{\mu}_2^T\boldsymbol{\Sigma}^{-1}\mathbf{x}) - \frac{1}{2}(\boldsymbol{\mu}_1^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1 - \mathbf{x}^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1 - \boldsymbol{\mu}_1^T\boldsymbol{\Sigma}^{-1}\mathbf{x}) + \log \frac{p(C_1)}{p(C_2)}. \end{aligned}$$

At this point it is worth emphasizing what is happening: although the Gaussian density involves a quadratic form $(\mathbf{x} - \boldsymbol{\mu}_k)^T\boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}_k)$, when the covariance is shared across classes, the quadratic terms in $\mathbf{x}$ cancel out in the log ratio, leaving a function that is *linear* in $\mathbf{x}$. This cancellation is precisely what makes the decision boundary linear in the shared-covariance GDA model.

Binary case. Observe that the results of all products involving $\boldsymbol{\Sigma}^{-1}$ are scalar, hence in particular

$$\begin{aligned} \mathbf{x}^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1 &= \boldsymbol{\mu}_1^T\boldsymbol{\Sigma}^{-1}\mathbf{x} \\ \mathbf{x}^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_2 &= \boldsymbol{\mu}_2^T\boldsymbol{\Sigma}^{-1}\mathbf{x}. \end{aligned}$$

Using these identities, we can regroup terms in $a(\mathbf{x})$ and obtain a linear discriminant:

$$a(\mathbf{x}) = \frac{1}{2}(\boldsymbol{\mu}_2^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1) + (\boldsymbol{\mu}_1^T\boldsymbol{\Sigma}^{-1} - \boldsymbol{\mu}_2^T\boldsymbol{\Sigma}^{-1})\boldsymbol{\phi}(\mathbf{x}) + \log \frac{p(C_1)}{p(C_2)} = \mathbf{w}^T\mathbf{x} + b.$$

The parameters $\mathbf{w}$ and b are explicitly given by

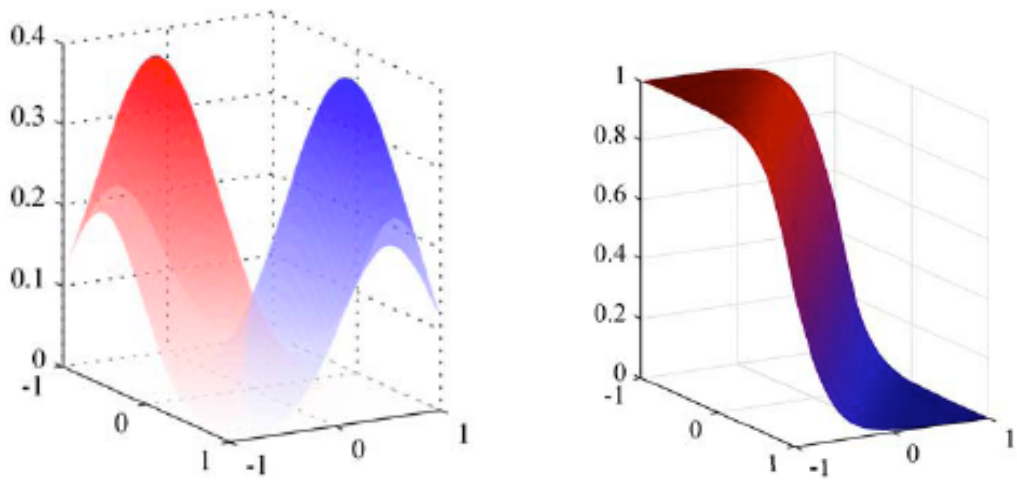
$$\begin{aligned} \mathbf{w} &= \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2) \\ b &= \frac{1}{2}(\boldsymbol{\mu}_2^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1) + \log \frac{p(C_1)}{p(C_2)}. \end{aligned}$$

Therefore, even though the model is derived from a generative probabilistic hypothesis, the posterior probability takes the form

$$p(C_1|\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + b),$$

which is computed by applying a non-linear function to a linear combination of the features. This is exactly the defining structure of a **generalized linear model**.

Example



Left: the class-conditional distributions $p(\mathbf{x}|C_1)$ and $p(\mathbf{x}|C_2)$, modeled as Gaussians in dimension $d = 2$. Right: the posterior probability $p(C_1|\mathbf{x})$, which exhibits the characteristic sigmoidal transition across the decision boundary.

Discriminant function

The discriminant function corresponds to the set of points $\mathbf{x}$ where the classifier is indifferent between the two classes, that is,

$$p(C_1|\mathbf{x}) = p(C_2|\mathbf{x}).$$

In terms of the sigmoid representation, this condition is equivalent to $\sigma(a(\mathbf{x})) = \sigma(-a(\mathbf{x}))$. Since the sigmoid is strictly increasing, equality implies equality of the arguments, so we must have $a(\mathbf{x}) = -a(\mathbf{x})$, hence $a(\mathbf{x}) = 0$. As a consequence, the decision boundary satisfies

$$\mathbf{w}^T\mathbf{x} + b = 0.$$

Replacing $\mathbf{w}$ and b with their GDA expressions yields

$$\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)\mathbf{x} + \frac{1}{2}(\boldsymbol{\mu}_2^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1) + \log\frac{p(C_2)}{p(C_1)} = 0.$$

This shows explicitly how the separating hyperplane depends not only on the class means and covariance, but also on the class priors: changing priors shifts the boundary by modifying the bias term.

A particularly transparent special case is $\boldsymbol{\Sigma} = \lambda\mathbf{I}$ (that is, $\sigma_{ii} = \lambda$ for $i = 1, \dots, d$). In this isotropic setting the discriminant becomes

$$2(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)\mathbf{x} + \|\boldsymbol{\mu}_1\|^2 - \|\boldsymbol{\mu}_2\|^2 + 2\lambda\log\frac{p(C_2)}{p(C_1)} = 0,$$

which highlights the role of the difference between the class means as the main geometric direction along which separation occurs.

Multiple classes

When $K > 2$, we return to the softmax posterior representation:

$$p(C_k|\mathbf{x}) = s(a_k(\mathbf{x})),$$

where $a_k(\mathbf{x}) = \log(p(\mathbf{x}|C_k)p(C_k))$. Under the shared-covariance Gaussian hypothesis, it follows by direct expansion of the Gaussian log-density that each $a_k(\mathbf{x})$ is affine in $\mathbf{x}$, and more precisely

$$a_k(\mathbf{x}) = \frac{1}{2}\left(\boldsymbol{\mu}_k^T\boldsymbol{\Sigma}^{-1}\mathbf{x} - \boldsymbol{\mu}_k^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_k\right) + \log p(C_k) - \frac{d}{2}\log(2\pi) - \frac{1}{2}\log|\boldsymbol{\Sigma}| = \mathbf{w}_k^T\mathbf{x} + b_k.$$

Thus, once again, the posterior probability

$$p(C_k|\mathbf{x}) = s(\mathbf{w}_k^T\mathbf{x} + b_k)$$

is computed by applying a non-linear function to a linear combination of the features, and the model retains the generalized linear structure.

To understand the geometry of the classifier, consider a point $\mathbf{x}$ where two classes C_j and C_k tie for the highest posterior probability:

$$p(C_k|\mathbf{x}) = p(C_j|\mathbf{x}) \qquad\qquad p(C_i|\mathbf{x}) < p(C_k|\mathbf{x}) \quad i \neq j, k.$$

Because the softmax is monotone in its arguments and preserves equality through exponentiation, this condition implies

$$e^{a_k(\mathbf{x})} = e^{a_j(\mathbf{x})} \qquad\qquad e^{a_i(\mathbf{x})} < e^{a^k(\mathbf{x})} \quad i \neq j, k,$$

hence

$$a_k(\mathbf{x}) = a_j(\mathbf{x}) \qquad\qquad a_i(\mathbf{x}) < a^k(\mathbf{x}) \quad i \neq j, k.$$

Since $a_k(\mathbf{x})$ and $a_j(\mathbf{x})$ are affine functions of $\mathbf{x}$, the equality $a_k(\mathbf{x}) = a_j(\mathbf{x})$ defines a hyperplane. Therefore, the decision boundaries in the shared-covariance multiclass GDA model are linear.

General covariance matrices, binary case

The linearity of the boundaries in the previous sections relies crucially on the shared-covariance hypothesis. If we relax this assumption and allow each class to have its own covariance matrix, then the cancellation of quadratic terms no longer occurs. In the binary case, with different covariance matrices $\boldsymbol{\Sigma}_1$ and $\boldsymbol{\Sigma}_2$, the log odds become

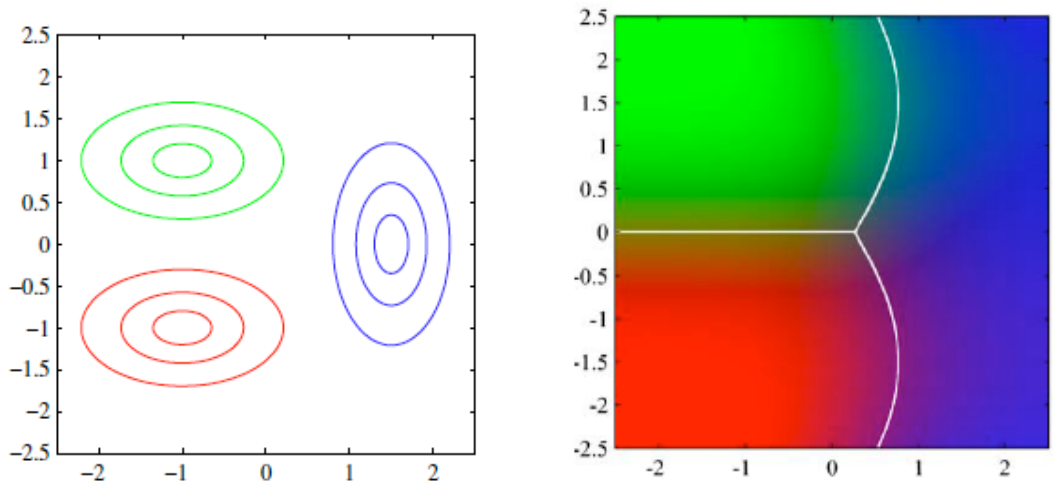
$$\begin{aligned} a(\mathbf{x}) &= \log\frac{p(\mathbf{x}|C_1)p(C_1)}{p(\mathbf{x}|C_2)p(C_2)} \\ &= \frac{1}{2}\left((\mathbf{x}-\boldsymbol{\mu}_2)^T\boldsymbol{\Sigma}_2^{-1}(\mathbf{x}-\boldsymbol{\mu}_2) - (\mathbf{x}-\boldsymbol{\mu}_1)^T\boldsymbol{\Sigma}_1^{-1}(\mathbf{x}-\boldsymbol{\mu}_1)\right) + \frac{1}{2}\log\frac{|\boldsymbol{\Sigma}_2|}{|\boldsymbol{\Sigma}_1|} + \log\frac{p(C_1)}{p(C_2)}. \end{aligned}$$

Applying the same decision rule as before (e.g. $a(\mathbf{x}) = 0$ for equal posteriors) yields the decision boundary

$$\left((\mathbf{x}-\boldsymbol{\mu}_2)^T\boldsymbol{\Sigma}_2^{-1}(\mathbf{x}-\boldsymbol{\mu}_2) - (\mathbf{x}-\boldsymbol{\mu}_1)^T\boldsymbol{\Sigma}_1^{-1}(\mathbf{x}-\boldsymbol{\mu}_1)\right) + \log\frac{|\boldsymbol{\Sigma}_2|}{|\boldsymbol{\Sigma}_1|} + 2\log\frac{p(C_1)}{p(C_2)} = 0.$$

In this setting, classes are separated by a (at most) quadratic surface. Intuitively, each Gaussian can have its own shape and orientation; the classifier then compares two quadratic forms, producing conic-section-like boundaries in low dimension and quadratic surfaces in higher dimension. More generally, it can be proved that the boundary surfaces are at most quadratic.

Example. Left: three classes modeled by Gaussians with different covariance matrices. Right: posterior distribution of classes, showing curved boundary surfaces.



GDA and maximum likelihood

So far we have treated the parameters $\boldsymbol{\mu}_k$, $\boldsymbol{\Sigma}$, and the priors $p(C_k)$ as known quantities. In practice they must be estimated from data. In the GDA framework this estimation can be carried out by maximum likelihood, yielding simple closed-form expressions that match our statistical intuition.

For the sake of simplicity, assume $K = 2$ and both classes share the same $\boldsymbol{\Sigma}$. We then need to estimate $\boldsymbol{\mu}_1, \boldsymbol{\mu}_2$, $\boldsymbol{\Sigma}$, and $\pi = p(C_1)$ (clearly, $p(C_2) = 1 - \pi$).

Training set $\mathcal{T}$: includes n elements $(\mathbf{x}_i, t_i)$, with

$$t_i = \begin{cases} 0 & \text{se } \mathbf{x}_i \in C_2 \\ 1 & \text{se } \mathbf{x}_i \in C_1 \end{cases}$$

If $\mathbf{x} \in C_1$, then

$$p(\mathbf{x}, C_1) = p(\mathbf{x}|C_1)p(C_1) = \pi \cdot \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_1, \boldsymbol{\Sigma}).$$

If $\mathbf{x} \in C_2$,

$$p(\mathbf{x}, C_2) = p(\mathbf{x}|C_2)p(C_2) = (1 - \pi) \cdot \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_2, \boldsymbol{\Sigma}).$$

The likelihood of the training set $\mathcal{T}$ is

$$L(\pi, \boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \boldsymbol{\Sigma}|\mathcal{T}) = \prod_{i=1}^{n} (\pi \cdot \mathcal{N}(\mathbf{x}_i|\boldsymbol{\mu}_1, \boldsymbol{\Sigma}))^{t_i} ((1 - \pi) \cdot \mathcal{N}(\mathbf{x}_i|\boldsymbol{\mu}_2, \boldsymbol{\Sigma}))^{1-t_i}.$$

Taking logs gives the log-likelihood

$$l(\pi, \boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \boldsymbol{\Sigma}|\mathcal{T}) = \sum_{i=1}^{n} \left(t_i \log \pi + t_i \log(\mathcal{N}(\mathbf{x}_i|\boldsymbol{\mu}_1, \boldsymbol{\Sigma}))\right) + \\ + \sum_{i=1}^{n} \left((1-t_i)\log(1-\pi) + (1-t_i)\log(\mathcal{N}(\mathbf{x}_i|\boldsymbol{\mu}_2, \boldsymbol{\Sigma}))\right).$$

Its derivative with respect to π is

$$\frac{\partial l}{\partial \pi} = \frac{\partial}{\partial \pi} \sum_{i=1}^{n} (t_i \log \pi + (1-t_i)\log(1-\pi)) = \sum_{i=1}^{n} \left(\frac{t_i}{\pi} - \frac{(1-t_i)}{1-\pi}\right) = \frac{n_1}{\pi} - \frac{n_2}{1-\pi},$$

which is equal to $\mathbf{0}$ for

$$\pi = \frac{n_1}{n}.$$

Thus, the maximum-likelihood estimate of the prior is simply the empirical class frequency, a result that aligns with the intuitive meaning of $p(C_1)$ as the probability of drawing a random training example from class C_1.

The maximum with respect to $\boldsymbol{\mu}_1$ (and similarly $\boldsymbol{\mu}_2$) is obtained by computing the gradient:

$$\nabla_{\boldsymbol{\mu}_1} l = \nabla_{\boldsymbol{\mu}_1} \sum_{i=1}^{n} t_i \log(\mathcal{N}(\mathbf{x}_i|\boldsymbol{\mu}_1, \boldsymbol{\Sigma})) = \boldsymbol{\Sigma}^{-1} \sum_{i=1}^{n} t_i(\mathbf{x}_i - \boldsymbol{\mu}_1).$$

As a consequence, we have $\nabla_{\boldsymbol{\mu}_1} l = \mathbf{0}$ for

$$\sum_{i=1}^{n} t_i \mathbf{x}_i = \sum_{i=1}^{n} t_i \boldsymbol{\mu}_1,$$

hence, for

$$\boldsymbol{\mu}_1 = \frac{1}{n_1} \sum_{\mathbf{x}_i \in C_1} \mathbf{x}_i.$$

Similarly, $\dfrac{\partial l}{\partial \boldsymbol{\mu}_2} = 0$ for

$$\boldsymbol{\mu}_2 = \frac{1}{n_2} \sum_{\mathbf{x}_i \in C_2} \mathbf{x}_i.$$

These expressions show that maximum likelihood recovers the sample means of each class, reinforcing the interpretation of $\boldsymbol{\mu}_k$ as the "center" of class C_k.

Maximizing the log-likelihood with respect to $\boldsymbol{\Sigma}$ provides

$$\boldsymbol{\Sigma} = \frac{n_1}{n} \mathrm{S}_1 + \frac{n_2}{n} \mathrm{S}_2,$$

where

$$\mathrm{S}_1 = \frac{1}{n_1} \sum_{\mathbf{x}_i \in C_1} (\mathbf{x}_i - \boldsymbol{\mu}_1)(\mathbf{x}_i - \boldsymbol{\mu}_1)^T$$
$$\mathrm{S}_2 = \frac{1}{n_2} \sum_{\mathbf{x}_i \in C_2} (\mathbf{x}_i - \boldsymbol{\mu}_2)(\mathbf{x}_i - \boldsymbol{\mu}_2)^T.$$

Thus the shared covariance estimate is a weighted average of the within-class covariance matrices, with weights given by the class proportions. This is consistent with the idea that $\boldsymbol{\Sigma}$ captures the typical spread of the data around the class means, pooled across classes.

From maximum likelihood to MAP estimation

These issues within the generative framework can be addressed also by adopting a Bayesian viewpoint and introducing prior distributions over the parameters. According to the MAP approach, rather than selecting parameters that only maximize the likelihood $p(\mathcal{T} \mid \theta)$, we select parameters that maximize the posterior distribution $p(\theta \mid \mathcal{T})$.

Let θ denote the collection of parameters of the generative model. In the shared-covariance binary GDA case, one may take

$$\theta = (\pi, \boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \boldsymbol{\Sigma}),$$

where $\pi = p(C_1)$. Bayes' rule gives the posterior over parameters as

$$p(\theta \mid \mathcal{T}) = \frac{p(\mathcal{T} \mid \theta)p(\theta)}{p(\mathcal{T})}.$$

Since $p(\mathcal{T})$ does not depend on θ, maximizing $p(\theta \mid \mathcal{T})$ is equivalent to maximizing the product $p(\mathcal{T} \mid \theta)p(\theta)$. Therefore, the MAP estimate is defined as

$$\hat{\theta}_{\text{MAP}} = \arg\max \theta \; p(\theta \mid \mathcal{T}) = \arg\max \theta \; p(\mathcal{T} \mid \theta)p(\theta).$$

Taking logs, MAP estimation can be expressed as an augmented optimization problem:

$$\hat{\theta}_{\text{MAP}} = \arg\max \theta \; \log p(\mathcal{T} \mid \theta) + \log p(\theta).$$

This expression makes again explicit the interpretation of MAP estimation as the inclusion to the log-likelihood of a a *regularization term* $\log p(\theta)$ that encodes prior beliefs about the parameters.

MAP estimation of the class prior.
Consider the estimation of $\pi = p(C_1)$ in the binary case. Under ML we obtained $\hat{\pi}_{\text{ML}} = n_1/n$. If instead we place a Beta prior $\pi \sim \text{Beta}(\alpha, \beta)$, the posterior is proportional to

$$p(\pi \mid \mathcal{T}) \propto \pi^{n_1}(1-\pi)^{n_2} \; \pi^{\alpha-1}(1-\pi)^{\beta-1} = \pi^{n_1+\alpha-1}(1-\pi)^{n_2+\beta-1}.$$

Maximizing this posterior yields a MAP estimate that can be seen as a smoothed version of the frequency estimator. In particular, the prior contributes *pseudo-counts* $\alpha - 1$ and $\beta - 1$ that prevent π from becoming too close to 0 or 1 when the dataset is small or unbalanced.

MAP estimation of covariance matrices.

A more substantial benefit arises for covariance estimation. In high-dimensional settings, the empirical (ML) covariance estimate can be ill-conditioned or even singular (for instance when d is not small compared to n), which makes $\boldsymbol{\Sigma}^{-1}$ numerically unstable or undefined. This is particularly problematic in GDA, because $\boldsymbol{\Sigma}^{-1}$ enters explicitly both in the discriminant direction $\mathbf{w} = \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)$ and in the bias term. When $\boldsymbol{\Sigma}$ is poorly estimated, small perturbations in the data can produce large changes in $\boldsymbol{\Sigma}^{-1}$ and therefore large swings in the decision boundary, harming generalization.

Introducing a Bayesian prior over $\boldsymbol{\Sigma}$ (for instance an inverse-Wishart prior in the Gaussian case) modifies the estimation criterion from pure likelihood maximization to posterior maximization. The practical consequence is that the MAP estimate behaves like a *regularized* covariance estimator: rather than trusting the sample covariance entirely, it blends the empirical information coming from the data with a structured reference covariance suggested by the prior. This blending is what is commonly called **shrinkage**.

To make the intuition concrete, it is useful to think of shrinkage in geometric terms. The empirical covariance typically has eigenvalues that are too extreme in finite-sample regimes: some directions in feature space appear spuriously "too variable" (large eigenvalues) while others appear spuriously "almost noiseless" (tiny eigenvalues). Inverting such a matrix amplifies exactly these errors: small eigenvalues become huge inverse-eigenvalues, and the classifier becomes overly sensitive along directions that were underestimated. A shrinkage/MAP estimator counteracts this effect by *pulling the eigenvalues toward a common scale* and by damping spurious correlations. In other words, the MAP estimate reduces anisotropy unless the data provide strong evidence that anisotropy is real. The result is a covariance matrix that is better conditioned, easier to invert, and typically yields more stable decision boundaries.

In the common "shrinkage-to-identity" interpretation, the regularized covariance can be seen (at least conceptually) as behaving like

$$\hat{\boldsymbol{\Sigma}}_{\mathrm{MAP}} \approx (1-\lambda)\,\hat{\boldsymbol{\Sigma}}_{\mathrm{ML}} + \lambda\,\tau\mathbf{I},$$

for some $\lambda \in [0,1]$ and scale $\tau > 0$. Although the exact form depends on the chosen prior and on the model assumptions, this expression captures the key effect: the MAP solution interpolates between the sample covariance and a simple, well-conditioned target (here isotropic noise). When λ is larger (typically when data are scarce), the estimate becomes closer to $\tau\mathbf{I}$, correlations are damped, and inversion becomes stable; when λ is small (typically with abundant data), the estimate approaches the ML covariance and the model becomes more flexible.

MAP estimation of means.

A closely related effect occurs for the class means. Under ML we obtained the intuitive estimators

$$\hat{\boldsymbol{\mu}}_k^{\mathrm{ML}} = \frac{1}{n_k} \sum_{\mathbf{x}_i \in C_k} \mathbf{x}_i,$$

namely the sample mean within each class. While these estimates are unbiased, they may have high variance when n_k is small, producing exaggerated differences between class centers and overly confident linear discriminants.

Under MAP, we introduce a prior distribution for $\boldsymbol{\mu}_k$. A standard choice is a Gaussian prior centered at some reference location $\boldsymbol{\mu}_0$,

$$\boldsymbol{\mu}_k \sim \mathcal{N}(\boldsymbol{\mu}_0,\ \boldsymbol{\Sigma}_0),$$

which expresses the belief that class means should not drift too far from $\boldsymbol{\mu}_0$ unless the data strongly support it. Maximizing the posterior then yields an estimator that *shrinks* the empirical class mean toward $\boldsymbol{\mu}_0$. In the simplest isotropic case $\boldsymbol{\Sigma}_0 = \kappa^{-1}\mathbf{I}$ (or more generally when the prior and likelihood are conjugate), the MAP estimate takes the form of a convex combination:

$$\hat{\boldsymbol{\mu}}_k^{\text{MAP}} = \alpha_k\, \hat{\boldsymbol{\mu}}_k^{\text{ML}} + (1 - \alpha_k)\, \boldsymbol{\mu}_0, \qquad \alpha_k \in (0, 1),$$

where α_k increases with the amount of data in class k (roughly, $\alpha_k \to 1$ as $n_k \to \infty$). This formula makes the effect transparent: with abundant data the MAP mean coincides with the sample mean, whereas with limited data the estimate is pulled toward the prior center. From the classifier's viewpoint, this reduces the risk of fitting noisy class centroids and typically yields more conservative, better-generalizing discriminant directions $\mathbf{w} = \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)$.

Overall, both for covariances and for means, MAP estimation can be interpreted as a principled mechanism for controlling variance in parameter estimates. By shrinking unstable empirical estimates toward structured prior targets, MAP improves numerical stability (especially through better-conditioned $\boldsymbol{\Sigma}$) and often improves predictive performance in finite-sample regimes, while retaining the probabilistic foundations of the generative model.

From a learning standpoint, the transition from ML to MAP does not modify the core classification rule—we still compute posteriors $p(C_k \mid \mathbf{x})$ via Bayes' rule using the learned parameters. What changes is the *estimation principle* used to determine those parameters. MAP introduces an explicit trade-off between fitting the observed data (through the likelihood) and adhering to prior assumptions (through the prior). When the amount of training data is large, the likelihood typically dominates and ML and MAP tend to coincide. When the dataset is small or the model is high-dimensional, MAP can provide a decisive

Conditional independence and Naive Bayes

The generative viewpoint introduced so far is not inherently tied to any specific type of feature representation. In fact, one of its main strengths is its flexibility: once a probabilistic model for the class-conditional distributions is specified, the same Bayes-based reasoning applies regardless of whether the features are continuous, discrete, or mixed. In this context, the **Naive Bayes** hypothesis should be understood as a structural assumption about feature dependencies.

As already shown, the Naive Bayes hypothesis states that, conditioned on the class label C_k, the features are mutually independent. Formally, for a generic feature vector $\mathbf{x} = (x_1, \dots, x_m)$, this assumption can be written as

$$p(\mathbf{x} \mid C_k) = \prod_{i=1}^{m} p(x_i \mid C_k).$$

Crucially, this factorization does *not* require the features to be discrete. Each marginal distribution $p(x_i \mid C_k)$ can belong to any parametric family appropriate for the domain of the i-th feature. For instance, if the features are continuous, one may assume Gaussian marginals,

$$p(x_i \mid C_k) = \mathcal{N}(x_i \mid \mu_{ki}, \sigma_{ki}^2),$$

leading to a class-conditional distribution that is a product of univariate Gaussians. From the perspective of Gaussian discriminant analysis, this corresponds to assuming that the covariance matrix $\boldsymbol{\Sigma}_k$ is diagonal. In other words, Naive Bayes applied to continuous features can be seen as a *restricted version of GDA*, in which correlations between different features are ignored.

This interpretation clarifies the effect of the Naive Bayes assumption on the model. By forcing the covariance matrix to be diagonal, the number of parameters is drastically reduced, which can be highly beneficial in high-dimensional settings or when the amount of training data is limited. At the same time, the model sacrifices the ability to capture correlations between features. Thus, Naive Bayes trades representational richness for statistical robustness, often yielding surprisingly good performance despite its seemingly strong independence assumption.

Discrete features and language models

Language models

At its most fundamental level, a **language model** can be defined as a categorical probability distribution established over a specific vocabulary of terms. This vocabulary is typically constructed from a large and representative collection of documents, capturing the statistical essence of how language is used within a particular domain or context.

The utility of a language model is twofold. First, it serves as a predictive engine: by understanding the distribution of terms, we can estimate the likelihood of a specific word occurring next in a sequence. Second, the probability of a term's occurrence is intrinsically linked to its **information content**. In the field of Information Retrieval, terms that are frequent across all documents are often seen as less discriminative, whereas rarer terms carry higher informational significance. This relationship forms the bedrock of modern indexing and retrieval algorithms.

A crucial simplification often made in basic language models is the **bag of words** hypothesis. Under this assumption, we treat the occurrence of each term as an independent event, completely decoupled from the terms that preceded it.

While this ignores the nuances of syntax, grammar, and sequential context, it allows for a computationally efficient representation of a document's "theme." Furthermore, once a model is derived, it can be used for **generative purposes**: by sampling repeatedly from the distribution, one can synthesize new documents that, while perhaps lacking coherent meaning, are statistically indistinguishable from the original collection in terms of word frequency.

Constructing the Model: From Corpus to Distribution

To move from a raw collection of text to a formal mathematical model, we follow a rigorous preparation and estimation process.

1. **Vocabulary Extraction and Pre-processing:** We define the dictionary $\mathcal{D} = \{t_1, \dots, t_n\}$ as the set of unique terms present in a given collection $\mathcal{C}$. This is not a simple list of every word found; rather, it is the result of two key operations:
   - **Stop word removal**: Eliminating common, high-frequency words (such as "the", "and", "is") that provide grammatical structure but carry little semantic weight.
   - **Stemming**: The process of reducing words to their linguistic root or basic form (e.g., "connected", "connecting", and "connection" all reduce to "connect"). This ensures that different morphological variations of the same concept are treated as the same term, effectively densifying the data.
2. **Calculating Multiplicities:** For each unique term t_i in our dictionary (where $i = 1, \dots, n$), we calculate its multiplicity m_i. This represents the total number of times the term t_i appears across the entire collection $\mathcal{C}$ after the aforementioned pre-processing steps.
3. **Deriving the Probability Vector:** The language model is expressed as a categorical distribution characterized by a probability vector $\hat{\boldsymbol{\phi}} = (\hat{\phi}_1, \dots, \hat{\phi}_n)^T$. Each component $\hat{\phi}_j$ represents the probability $p(t_j|\mathcal{C})$ of encountering term t_j when randomly picking a word from the collection.

To ensure the model is mathematically sound as a probability distribution, the vector must satisfy the following constraints:

$$0 \leq \hat{\phi}_i \leq 1 \quad \text{for all } i = 1, \dots, n \qquad\qquad \sum_{i=1}^{n} \hat{\phi}_i = 1$$

The value $\hat{\phi}_i$ is essentially the **Maximum Likelihood Estimate (MLE)** for the probability of term t_i in a multinomial setting. If we denote the total number of term occurrences in the processed collection as $N = \sum_{j=1}^{n} m_j$, then the most intuitive way to derive the parameters is via the observed frequency:

$$\hat{\phi}_i = \frac{m_i}{N}$$

This reflects the "frequentist" view of language modeling: the best estimate for the probability of a word is simply how often we have observed it in the past. This vector $\hat{\boldsymbol{\phi}}$ becomes the identity of the corpus, allowing us to compare different collections of text by measuring the distance between their respective probability vectors.

Handling Data Sparsity: Laplace Smoothing

The Maximum Likelihood Estimate (MLE) for term probabilities, $\hat{\phi}_i = \frac{m_i}{N}$, is highly efficient but suffers from a critical flaw: the **zero-frequency problem**. If a term t_{new} exists in the language but happens to be absent from the training corpus $\mathcal{C}$ ($m_{new} = 0$), the model assigns it a probability of zero. In generative or predictive tasks, this implies that the occurrence of this term is an "impossible event," which is a poor reflection of reality.

To mitigate this, we apply **smoothing** (also known as regularization in this context). The goal is to redistribute some of the probability mass from frequent terms to terms with zero or very low counts, effectively ensuring that no term is ever considered truly impossible.

The simplest and most historically significant smoothing method is **Laplace Smoothing**, or "add-one" smoothing. The underlying idea is to pretend we have seen every possible term in the dictionary at least once more than we actually have.

Given a dictionary $\boldsymbol{D}$ of size n and a total count of terms $N = \sum m_i$, the smoothed probability ϕ_i^{Lap} is calculated as:

$$\phi_i^{Lap} = \frac{m_i + 1}{N + n}$$

The addition of 1 to each numerator and n to the denominator ensures that the distribution remains normalized:

$$\begin{aligned}\sum_{i=1}^{n} \phi_i^{Lap} &= \sum_{i=1}^{n} \frac{m_i + 1}{N + n} = \frac{\sum m_i + \sum 1}{N + n} \\ &= \frac{N + n}{N + n} = 1\end{aligned}$$

We can interpret Laplace smoothing through several lenses:

- From a Bayesian perspective (see below), Laplace smoothing is equivalent to assuming a **Dirichlet prior** on the categorical distribution where all hyper-parameters $\alpha_i = 1$. This represents a prior belief that all words are equally likely before any data is observed.
- We are effectively adding "pseudo-counts" to our observations. For very large datasets ($N \gg n$), the effect of the $+1$ is negligible, and the distribution converges to the MLE. However, for small datasets or rare terms, the smoothing provides a vital "safety net."
- By assigning a non-zero probability $\frac{1}{N+n}$ to unseen words, we allow Information Retrieval systems to calculate finite weights for all possible queries, preventing the system from failing when faced with novel vocabulary.

Laplace smoothing can be generalized by adding a parameter α (where $0 < \alpha \leq 1$) instead of a literal 1. This is known as **Lidstone smoothing**:

$$\phi_i^{(\alpha)} = \frac{m_i + \alpha}{N + \alpha n}$$

Choosing a small α (e.g., $\alpha = 0.1$) allows for a less aggressive redistribution of probability mass, which is often preferred in large-scale modern natural language processing to avoid over-estimating the likelihood of extremely rare words.

A Bayesian Perspective on Language Modeling

In the previous sections, we explored the frequentist approach to language modeling, which relies primarily on observed counts. However, this approach often fails when data is sparse. To build a more robust model, we can adopt a **Bayesian framework**, treating the parameters of our language model not as fixed constants, but as random variables characterized by a prior belief.

The Dirichlet-Multinomial Model

When modeling text as a sequence of independent terms (the bag-of-words assumption), we are essentially dealing with a multinomial process. In Bayesian statistics, the **Dirichlet distribution** is the natural choice for a prior because it is the *conjugate prior* to the multinomial distribution. This conjugacy is mathematically convenient: it ensures that if our prior is Dirichlet, our posterior will also be Dirichlet.

The process begins by defining a **Dirichlet prior** $\mathrm{Dir}(\boldsymbol{\phi}|\boldsymbol{\alpha})$, governed by the hyper-parameter vector $\boldsymbol{\alpha} = (\alpha_1, \alpha_2, \dots, \alpha_n)$. The probability density function is given by:

$$p(\phi_1, \dots, \phi_n|\boldsymbol{\alpha}) = \frac{1}{\Delta(\alpha_1, \dots, \alpha_n)} \prod_{i=1}^{n} \phi_i^{\alpha_i - 1}$$

where $\Delta(\boldsymbol{\alpha})$ is the multivariate Beta function, acting as the normalizing constant. These α_i parameters can be interpreted as "prior observations" or "pseudo-counts"—essentially, our belief about how many times each word has occurred before we have seen any actual documents.

As we observe a collection of documents $\boldsymbol{C}$, we record the multiplicities m_i for each term t_i. In the Bayesian logic, we "update" our prior distribution with this new evidence. Because of the conjugacy mentioned earlier, the **posterior distribution** of $\boldsymbol{\phi}$ after observing $\boldsymbol{C}$ remains a Dirichlet distribution, denoted as $\mathrm{Dir}(\boldsymbol{\phi}|\overline{\boldsymbol{\alpha}})$, with the updated hyper-parameters:

$$\overline{\boldsymbol{\alpha}} = (\alpha_1 + m_1, \alpha_2 + m_2, \dots, \alpha_n + m_n)$$

This update is remarkably intuitive: we simply add the observed counts m_i to our prior pseudo-counts α_i. The resulting posterior density is:

$$p(\phi_1, \dots, \phi_n|\overline{\boldsymbol{\alpha}}) = \frac{1}{\Delta(\alpha_1 + m_1, \dots, \alpha_n + m_n)} \prod_{i=1}^{n} \phi_i^{\alpha_i + m_i - 1}$$

Rather than picking a single point estimate for the probabilities (like the mode or the mean), a full Bayesian treatment determines the **predictive posterior distribution**. We want to find the probability of the next term t_j given the history $\boldsymbol{C}$ and our prior $\boldsymbol{\alpha}$. This requires integrating over all possible values of the parameter vector $\boldsymbol{\phi}$:

$$\begin{aligned}
\hat{\phi}_j = p(t_j|\mathcal{C}, \boldsymbol{\alpha}) &= \int p(t_j|\boldsymbol{\phi}) p(\boldsymbol{\phi}|\mathcal{C}, \boldsymbol{\alpha}) d\boldsymbol{\phi} \\
&= \int \phi_j \mathrm{Dir}(\boldsymbol{\phi}|\overline{\boldsymbol{\alpha}}) d\boldsymbol{\phi} \\
&= \mathop{\mathbb{E}}_{\boldsymbol{\phi} \sim \mathrm{Dir}(\cdot|\overline{\boldsymbol{\alpha}})} \left[\phi_j\right]
\end{aligned}$$

Mathematically, the probability of term t_j in our language model corresponds to the **expected value** of ϕ_j with respect to the posterior Dirichlet distribution. By the properties of the Dirichlet distribution, the mean for the j-th component is:

$$\hat{\phi}_j = \frac{\overline{\boldsymbol{\alpha}}_j}{\sum_{k=1}^{n} \overline{\boldsymbol{\alpha}}_k} = \frac{\alpha_j + m_j}{\sum_{k=1}^{n}(\alpha_k + m_k)} = \frac{\alpha_j + m_j}{\alpha_0 + N}$$

where $\alpha_0 = \sum \alpha_k$ is the total mass of the prior and N is the total number of words in the corpus.

This result is fundamental because of **Dirichlet smoothing**. As long as $\alpha_j > 0$, the probability $\hat{\phi}_j$ can never be zero, even if the word t_j never appeared in the corpus ($m_j = 0$). This effectively solves the zero-frequency problem by "smoothing" the distribution.

If we have no prior reason to believe one word is more likely than another, we use a **non-informative prior** where $\alpha_i = \alpha$ for all i. In this symmetric case, the probability simplifies to:

$$p(t_j|C, \boldsymbol{\alpha}) = \frac{m_j + \alpha}{\alpha|\boldsymbol{D}| + N}$$

where $|\boldsymbol{D}|$ represents the size of the vocabulary (the number of unique terms).

Observe that when $\alpha = 1$, this formula becomes identical to the Laplace (add-one) smoothing introduced previously. Thus, Laplace smoothing can be formally interpreted as a Bayesian inference process where we start with a uniform prior that assumes every word has already been seen exactly once.

Naive bayes classifiers

Language models, which we have previously defined as generative tools for understanding word distributions, find one of their most powerful applications in the field of supervised learning, specifically for document classification. By modeling how each class "generates" language, we can reverse the process to determine which class a specific document most likely belongs to.

Suppose we are faced with a binary classification problem involving two classes, C_1 and C_2 (for example, "Spam" vs. "Ham" or "Positive" vs. "Negative" sentiment). For any given document d, our goal is to compute the posterior probabilities $p(C_1|d)$ and $p(C_2|d)$. Once these are known, a common decision rule is to assign the document to the class that yields the higher probability.

To derive these posterior probabilities from a collection of labeled data - namely, a set $\boldsymbol{C}_1$ of documents belonging to C_1 and a set $\boldsymbol{C}_2$ belonging to C_2 - we rely on **Bayes' rule**:

$$p(C_k|d) = \frac{p(d|C_k)p(C_k)}{p(d)} \propto p(d|C_k)p(C_k)$$

In this context, the evidence $p(d)$ acts as a normalizing constant. Since it is identical for all classes, it does not affect the comparison between $p(C_1|d)$ and $p(C_2|d)$ and can therefore be ignored during the ranking process. This leaves us with two main components to estimate from our training collections:

1. The prior probability $p(C_k)$, representing our belief in the class frequency before looking at the document.
2. The class-conditional likelihood $p(d|C_k)$, representing how likely the class C_k is to generate the specific sequence of words found in d.

The estimation of the priors is straightforward. Using a Maximum Likelihood (ML) approach, we simply look at the relative proportions of the classes in our training set:

$$p(C_k) = \frac{|C_k|}{|C_1| + |C_2|}$$

This estimate assumes that our training collections are representative of the true distribution of classes in the real world.

The estimation of the likelihood $p(d|C_k)$ is more complex. According to the *bag of words* assumption, we treat the document d as a multiset (or "bag") of n_d terms:

$$d = \{\bar{t}_1, \bar{t}_2, \dots, \bar{t}_{n_d}\}$$

If we were to apply the general product rule of probability, the likelihood would be:

$$\begin{aligned} p(d|C_k) &= p(\bar{t}_1, \dots, \bar{t}_{n_d}|C_k) \\ &= p(\bar{t}_1|C_k)p(\bar{t}_2|\bar{t}_1, C_k) \cdots p(\bar{t}_{n_d}|\bar{t}_1, \dots, \bar{t}_{n_d-1}, C_k) \end{aligned}$$

Calculating these conditional dependencies is computationally prohibitive and requires an enormous amount of data, as most specific word sequences will never appear in the training set.

The Naive Bayes Assumption

To make this calculation feasible, we introduce the **Naive Bayes assumption**. We assume that the terms in a document are **pairwise conditionally independent** given the class C_k. Mathematically, for any two terms i and j, this means:

$$p(\bar{t}_i, \bar{t}_j|C_k) = p(\bar{t}_i|C_k)p(\bar{t}_j|C_k)$$

While this assumption is "naive" because it ignores the obvious linguistic dependencies (e.g., the word "San" makes the word "Francisco" much more likely), it dramatically simplifies the likelihood into a simple product of independent probabilities:

$$p(d|C_k) = \prod_{j=1}^{n_d} p(\bar{t}_j|C_k)$$

The individual probabilities $p(\bar{t}_j|C_k)$ are exactly the values provided by the **language models** we have derived for each class C_k. By applying smoothing (such as Laplace smoothing) to these individual term probabilities, we ensure that the entire product does not vanish if a single word in d was missing from the training set of class C_k.

By combining these components, the Naive Bayes classifier operates through the following algorithmic steps to classify a new document d:

1. Represent document d as a bag of words $\bar{t}_1, \dots, \bar{t}_{n_d}$.
2. For each class $k \in \{1, 2\}$, retrieve the term probabilities $p(\bar{t}_i|C_k)$ from the respective language models.

3. For each class, calculate the joint likelihood $p(d|C_k)$ by multiplying the individual term probabilities together.
4. Assign d to the class C_r that maximizes the product of the likelihood and the prior:

$$r = \underset{k \in \{1,2\}}{\operatorname{argmax}}\, p(d|C_k)p(C_k)$$

It is important to note that this generative approach is not restricted to text. It can be applied to any item $\mathbf{x} = (x_0, \dots, x_d)$ described by a vector of features. In the general case, the Naive Bayes assumption states that all features are conditionally independent given the class, allowing us to model the joint distribution as:

$$p(\mathbf{x}|C_k) = \prod_{i=0}^{d} p(x_i|C_k)$$

Despite its simplicity and the theoretical "incorrectness" of the independence assumption, Naive Bayes often performs remarkably well, particularly when the features are highly dimensional.

Feature selection by mutual information

The set of probabilities in a language model can be exploited to identify the most relevant terms for classification,[1] that is terms whose presence or absence in a document best characterizes the class of the document.

To measure relevance, we can apply the set of mutual informations $\{I_1, \dots, I_n\}$

$$\begin{aligned} I_j &= \sum_{k=1,2} p(t_j, C_k) \log \frac{p(t_j, C_k)}{p(t_j)p(C_k)} \\ &= \sum_{k=1,2} p(C_k|t_j)p(t_j) \log \frac{p(C_k|t_j)}{p(C_k)} = p(t_j)KL(p(C_k|t_j)||p(C_k)) \end{aligned}$$

here, KL is a measure of the amount of information on class distributions provided by the presence of t_j. This amount is weighted by the probability of occurrence of t_j.

Since $p(t_j, C_k) = p(C_k|t_j)p(t_j) = p(t_j|C_k)p(C_k)$, I_j can be estimated as

$$\begin{aligned} I_j &= p(t_j|C_1)p(C_1) \log \frac{p(t_j|C_1)}{p(t_j)} + p(t_j|C_2)p(C_2) \log \frac{p(t_j|C_2)}{p(t_j)} \\ &= \phi_{j1}\pi_1 \log \frac{\phi_{j1}}{\phi_{j1}\pi_1 + \phi_{j2}\pi_2} + \phi_{j2}\pi_2 \log \frac{\phi_{j2}}{\phi_{j1}\pi_1 + \phi_{j2}\pi_2} \end{aligned}$$

where ϕ_{jk} is the estimated probability of t_j in documents of class C_k and π_k is the estimated probability of a document of class C_k in the collection.

A selection of the most significant terms can be performed by selecting the set of terms with highest mutual information I_j.

[1] As done before, these considerations can be extended to any set of features. Just consider the number of occurrences of a term as a particular type of feature.

Feature Selection via Mutual Information

In high-dimensional tasks such as document classification, we often encounter thousands of unique terms. However, not all terms are equally informative; some are generic, while others are highly specific to a particular category. **Feature selection** is the process of identifying a subset of the most relevant terms to improve computational efficiency and model generalization. While these considerations are framed within the context of language models, they can be naturally extended to any set of features by treating the presence or multiplicity of a feature similarly to the occurrence of a term.

The goal is to identify terms whose presence or absence in a document provides the strongest "signal" regarding the document's class. A mathematically rigorous way to quantify this signal is through the concept of **Mutual Information**.

To measure the relevance of a term t_j, we examine the Mutual Information I_j between the distribution of that term and the distribution of the classes C_k. This metric quantifies how much uncertainty about the class is reduced by knowing whether the term t_j is present. We define the set of mutual informations $\{I_1, \dots, I_n\}$ as:

$$
\begin{aligned}
I_j &= \sum_{k=1,2} p(t_j, C_k) \log \frac{p(t_j, C_k)}{p(t_j)p(C_k)} \\
&= \sum_{k=1,2} p(C_k|t_j)p(t_j) \log \frac{p(C_k|t_j)}{p(C_k)} \\
&= p(t_j) \sum_{k=1,2} p(C_k|t_j) \log \frac{p(C_k|t_j)}{p(C_k)} \\
&= p(t_j) KL(p(C_k|t_j) \| p(C_k))
\end{aligned}
$$

The final derivation above provides a deep conceptual insight: Mutual Information is the **Kullback-Leibler (KL) Divergence** between the conditional distribution of the classes given the term, $p(C_k|t_j)$, and the prior distribution of the classes, $p(C_k)$.

- The KL term measures the "distance" or divergence between our knowledge of the class *after* seeing term t_j versus *before* seeing it.
- If t_j is non-informative (like a stop word), the distribution $p(C_k|t_j)$ will be almost identical to $p(C_k)$, resulting in a KL close to zero.
- This divergence is weighted by $p(t_j)$, the probability of the term's occurrence. This ensures that we do not over-prioritize extremely rare terms that might show a high divergence purely by chance but almost never appear in the data.

In practice, we estimate I_j using the parameters derived from our training collections. By applying the identity $p(t_j, C_k) = p(t_j|C_k)p(C_k)$, we can rewrite the mutual information in terms of our language model parameters:

$$I_j = p(t_j|C_1)p(C_1)\log\frac{p(t_j|C_1)}{p(t_j)} + p(t_j|C_2)p(C_2)\log\frac{p(t_j|C_2)}{p(t_j)}$$

By substituting the estimated parameters, we obtain a formula directly applicable to our data:

$$I_j = \phi_{j1}\pi_1\log\frac{\phi_{j1}}{\phi_{j1}\pi_1 + \phi_{j2}\pi_2} + \phi_{j2}\pi_2\log\frac{\phi_{j2}}{\phi_{j1}\pi_1 + \phi_{j2}\pi_2}$$

Where:

- $\phi_{jk} = p(t_j|C_k)$ is the estimated probability of term t_j occurring in documents of class C_k (the language model parameter).
- $\pi_k = p(C_k)$ is the estimated prior probability of a document belonging to class C_k in the collection.
- The denominator $\phi_{j1}\pi_1 + \phi_{j2}\pi_2$ represents the total marginal probability $p(t_j)$ of the term occurring across the entire collection.

Once I_j has been computed for every term in the vocabulary, the selection process is straightforward: we rank the terms in descending order of their mutual information values. We then select the top K terms, where K is a hyper-parameter chosen based on the desired dimensionality of the reduced feature space.

This subset of terms represents the "most significant" features. By focusing only on these terms, the Naive Bayes classifier (or any other classification model) becomes more robust to noise and significantly faster to train and evaluate, as it ignores terms that do not contribute meaningful information for distinguishing between the classes.

Comparing Mutual Information vs. Frequency-based Selection

In feature selection, two of the most common approaches are ranking by **Global Frequency** (choosing the most common words) and ranking by **Mutual Information** (choosing the most discriminative words). While frequency is easier to compute, it often fails to capture the most useful features for classification.

Frequency-based selection identifies the most common terms across the entire collection $\boldsymbol{C}$, regardless of class distribution. This often leads to the selection of:

- Stop words: Even after standard removal, common words like "time," "people," or "system" may dominate the frequency list.
- Domain-Specific Background Words: In a medical corpus, the word "patient" will be highly frequent but may appear equally in "Oncology" and "Cardiology" documents, providing no help in distinguishing between them.

Mutual Information (MI), instead, prioritizes terms that have a strong *correlation* with a specific class. A word earns a high MI score if it appears frequently in one class but rarely in others.

Consider a binary classification task between **Sports** (C_1) and **Politics** (C_2).

Term	Freq. in Sports	Freq. in Politics	Selection Bias	Informative Power
"the"	Extremely High	Extremely High	Low MI	None (Noise)
"said"	High	High	Low MI	None (Background)
"touchdown"	High	Near Zero	**High MI**	Strong (Discriminative)
"veto"	Near Zero	Medium	**High MI**	Strong (Discriminative)

The mathematical transition from frequency to information reveals three key differences:

1. Frequency-based selection captures the "skeleton" of the language. Mutual Information captures the "lexicon of the topic." MI naturally ignores high-frequency words that are evenly distributed across classes.
2. As we saw in the formula $I_j = p(t_j)KL(p(C_k|t_j)\|p(C_k))$, the $p(t_j)$ term acts as a filter. Unlike purely discriminative measures (like simple probability ratios), MI requires a term to be *somewhat* frequent to be considered relevant, preventing the selection of "noise" terms that appear only once in the entire corpus.
3. By selecting terms with high MI, the Naive Bayes classifier focuses its product $\prod p(t_j|C_k)$ on terms where the ratio $\frac{p(t_j|C_1)}{p(t_j|C_2)}$ is significantly different from 1. This amplifies the "gap" between class probabilities, leading to more confident and accurate classifications.

Conclusions.

- Frequency is best when the goal is *Language Modeling* or *Compression* (summarizing the general content).
- Mutual Information is superior for *Supervised Learning* and *Classification*, as it explicitly searches for the boundary between categories.